ITS Villada

Base de Datos II

INFORME DE INVESTIGACIÓN

Grupo N° 8

Procesamiento distribuido y *machine learning* sobre datos masivos con Apache Spark MLlib

Modelo de Datos, Arquitectura Interna, Algoritmos Iterativos y Optimización

Integrantes

Leon Guchea
Bautista Daveloze
Riccioni Santiago
Sibona Luciano
Macias Luciano

Docentes

Lisandro Márquez
Teodoro Reyna

11 de noviembre de 2026

***Resumen*—Cuando el volumen de un conjunto de datos excede las capacidades físicas de un único servidor, su tratamiento exige estrategias de particionamiento horizontal y cómputo paralelo en clúster. Este informe analiza integralmente el procesamiento distribuido moderno en Apache Spark y su biblioteca de aprendizaje automático, MLlib. Se examina formalmente el modelo de datos (RDDs, DataFrames estructurados bajo esquemas *StructType* y formatos columnares particionados en disco como Parquet), la arquitectura interna de ejecución (distribución por particiones, ausencia deliberada de replicación en memoria y resiliencia por linaje determinista, técnicas de aceleración por poda y filtros de Bloom en lugar de índices B-Tree, y consistencia mediante protocolos de confirmación atómica y capas transaccionales). Asimismo, se establece una comparación fundamentada con el modelo relacional tradicional, delimitando casos de uso analíticos reales y escenarios donde no conviene utilizar la tecnología. Finalmente, se presentan ejemplos de operaciones concretas (consultas declarativas, escrituras particionadas y canalizaciones completas de ML), formalizando la paralelización del algoritmo K-Means con agregación jerárquica en árbol (*treeAggregate*) y las optimizaciones de bajo nivel de Catalyst y Project Tungsten.**

***Palabras clave*—Apache Spark, MLlib, modelo de datos, arquitectura distribuida, DataFrames, RDD, consistencia, indexación, modelo relacional, K-Means, Catalyst, Project Tungsten, treeAggregate.**

I. Introducción

El crecimiento exponencial del volumen de datos en las organizaciones modernas (registros de transacciones financieras, eventos de interacción de usuarios, telemetría IoT y flujos continuos en tiempo real) ha superado la capacidad de procesamiento de las arquitecturas tradicionales basadas en un único servidor. Frente a este desafío, existen dos alternativas de escalamiento: vertical (*scale-up*), incrementando la memoria y procesadores de una máquina central, u horizontal (*scale-out*), coordinando clústeres de computadoras conectadas en red. La vía horizontal resulta más económica y no posee un límite físico estricto, pero introduce retos complejos de sistemas distribuidos: estructuración y particionamiento de datos, coordinación inter-nodo, tráfico de red (*shuffle*) y tolerancia a fallos parciales de hardware.

El paradigma MapReduce [1], formalizado por Google y popularizado mediante Apache Hadoop y su sistema de archivos distribuido HDFS [2], resolvió la tolerancia a fallos en transformaciones masivas por lotes mediante la persistencia y replicación física de resultados intermedios en disco. No obstante, este enfoque demostró una ineficiencia severa en algoritmos iterativos característicos del aprendizaje automático (*Machine Learning*) y en consultas analíticas interactivas, donde los mismos conjuntos de datos se exploran decenas de veces. Apache Spark [3], [4] fue concebido para superar este obstáculo, reteniendo los datos en la memoria RAM de los nodos trabajadores entre etapas sucesivas y sustituyendo la replicación costosa por un grafo de linaje lógico determinista.

El presente informe tiene como propósito abordar los aspectos fundamentales exigidos para la evaluación de tecnologías distribuidas:

1) **Modelo de datos**: cómo se estructuran (RDD, DataFrame, Dataset), cómo se almacenan en memoria y almacenamiento columnar en disco, y cómo se consultan.
2) **Arquitectura y funcionamiento interno**: mecanismos de distribución de tareas, políticas de replicación, estrategias de indexación/poda de datos y modelos de consistencia de lectura y escritura.
3) **Comparación fundamentada con el modelo relacional**: contraste de paradigmas (OLTP frente a OLAP, normalización frente a desnormalización, ACID frente a consistencia de lotes) y complementariedad en arquitecturas modernas.
4) **Casos de uso reales y contraindicaciones**: escenarios analíticos donde la tecnología destaca y situaciones donde su uso degrada el rendimiento.
5) **Operaciones concretas**: ejemplos de código ejecutable que abarcan consultas analíticas, escrituras particionadas, entrenamiento iterativo de K-Means y canalizaciones completas (*ML Pipelines*).

El resto del informe se estructura así: la Sección II detalla el modelo de datos, la arquitectura interna, replicación, indexación y consistencia; la Sección III profundiza en la comparación con el modelo relacional, casos de uso reales y contraindicaciones; la Sección IV presenta operaciones concretas de consulta/escritura y el caso de estudio de K-Means; la Sección V analiza la tolerancia a fallos, mitigación de rezagados y rendimiento comparado frente a Hadoop; la Sección VI formaliza el aprendizaje automático a escala, agregación en árbol, canalizaciones y optimización del motor (Catalyst y Tungsten); y la Sección VII sintetiza las conclusiones.

II. Modelo de datos, arquitectura y funcionamiento interno

A. Modelo de datos: Estructura, almacenamiento y consulta

A diferencia de los sistemas de bases de datos tradicionales que imponen un único motor de almacenamiento y un esquema relacional estricto, Apache Spark opera como un motor de procesamiento desacoplado que define tres abstracciones sucesivas de modelado de datos:

1) Resilient Distributed Datasets (RDD): Colección distribuida de objetos inmutable, particionada y evaluada de forma perezosa (*lazy evaluation*) [4]. Los RDD operan a bajo nivel sin requerir un esquema predefinido; pueden almacenar objetos arbitrarios (estructurados, semiestructurados o texto libre). Las operaciones sobre RDD se dividen en:

- *Transformaciones* (`map`, `filter`, `groupByKey`): Crean un nuevo RDD a partir del anterior sin materializarlo, registrando la operación en un grafo dirigido acíclico (DAG).
- *Acciones* (`count`, `collect`, `saveAsTextFile`): Disparan la ejecución y optimización física del DAG para devolver un resultado al *driver* o escribirlo en almacenamiento.

2) DataFrames y Datasets: Un DataFrame es una colección distribuida organizada en columnas con nombre y tipos de datos bien definidos, equivalente conceptual a una tabla relacional pero optimizada para cómputo distribuido [15]. Su estructura se rige por un esquema explícito denominado `StructType`, compuesto por múltiples campos `StructField`. Los *Datasets* (disponibles en Scala y Java)

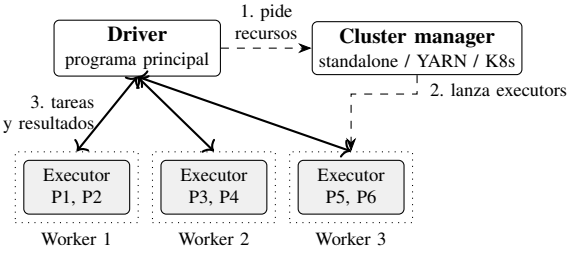


Fig. 1. Arquitectura de ejecución en Apache Spark. Tras la asignación de recursos por el Cluster Manager, el Driver despacha tareas directamente a los Executors y recolecta resultados.

agregan seguridad de tipos en tiempo de compilación (*compile-time type safety*).

3) Almacenamiento físico en memoria y disco:

- *En memoria RAM*: Spark almacena los registros de DataFrames utilizando el formato binario compacto `UnsafeRow` de Project Tungsten. Los datos se ubican en bloques contiguos de memoria fuera del recolector de basura (*Off-Heap*), suprimiendo la sobrecarga de encabezados de objetos Java y evitando pausas de recolección (*Garbage Collection pauses*).
- *En almacenamiento persistente*: Spark no posee un formato de disco propietario; se integra con sistemas distribuidos (HDFS, Amazon S3, Google Cloud Storage). El almacenamiento óptimo es de tipo **columnar particionado** (Apache Parquet o Apache ORC), comprimido con algoritmos como Snappy o Zstandard. Los datos se organizan en jerarquías de directorios que codifican columnas de partición (ej. `/ventas/anio=2026/mes=11/`).

4) Mecanismos de consulta: Los datos se consultan indistintamente mediante la API declarativa de DataFrames (métodos encadenables `select`, `filter`, `groupBy`, `join`) o mediante sentencias estándar ANSI SQL (`spark.sql("SELECT ...")`) registradas sobre tablas o vistas temporales. Ambas APIs se unifican bajo el mismo compilador lógico Catalyst [11].

B. Arquitectura y distribución del cómputo

La arquitectura de una aplicación Spark sigue el modelo maestro–trabajador (*master-worker*), ilustrado en la Fig. 1:

- **Driver**: Proceso principal que ejecuta el método `main`. Transforma el código en un DAG de dependencias, divide el DAG en etapas (*stages*) según los límites de *shuffle*, programa las tareas individuales (*tasks*) y consolida los resultados.
- **Cluster Manager**: Administrador de recursos del clúster (Standalone, Apache Hadoop YARN [7] o Kubernetes). Asigna nodos y memoria a la aplicación; no interviene en el cómputo de datos.
- **Worker Nodes y Executors**: Servidores físicos o pods que alojan procesos *executors*. Cada ejecutor ejecuta tareas en múltiples hilos sobre particiones de datos y conserva datos persistidos en memoria.

Distribución de datos: La unidad atómica de paralelismo es la **partición**. Los datos se distribuyen entre particiones mediante particionamiento por dispersión (`HashPartitioner`, aplicando `hash(clave) % numParticiones`) o por rangos ordenados (`RangePartitioner`). Las transformaciones se clasifican en:

1) *Dependencias estrechas* (*narrow*): Cada partición hija depende de una única partición padre (ej. `map`, `filter`). Se procesan en paralelo dentro del mismo ejecutor (*pipelining*) sin tráfico de red.
2) *Dependencias amplias* (*wide*): Múltiples particiones hijas requieren datos de distintas particiones padres (ej. `groupByKey`, `join`). Requieren redistribución global por red (*shuffle*) y marcan las fronteras entre etapas.

C. Replicación, linaje y resiliencia

A diferencia de los sistemas de almacenamiento como HDFS, que garantizan tolerancia a fallos manteniendo tres réplicas físicas de cada bloque en disco, Spark adopta una filosofía radicalmente distinta: **no replica particiones en memoria por defecto** (factor de replicación 1). Replicar en memoria RAM terabytes de datos intermedios reduciría a la mitad o a un tercio la memoria disponible para el cómputo real.

En su lugar, Spark implementa tolerancia a fallos mediante el **linaje determinista del DAG** [4]:

- Cada RDD o DataFrame registra la receta de operaciones y las particiones padre de las que proviene.
- Si un ejecutor falla y pierde una o más particiones en RAM, el *driver* detecta la ausencia de la tarea y la despacha a otro ejecutor disponible.
- El nuevo ejecutor reconstruye *únicamente la partición perdida*, leyendo el fragmento original desde el almacenamiento base y reaplicando las transformaciones registradas.

Para escenarios críticos donde recomputar una partición resulta sumamente costoso, el programador puede definir niveles de persistencia avanzados mediante `persist(StorageLevel.MEMORY_AND_DISK_2)`, lo que instruye a Spark a replicar particiones en dos nodos trabajadores diferentes. La persistencia definitiva de los datos finales se delega al almacenamiento duradero subyacente (HDFS, S3).

D. Mecanismos de indexación y modelo de consistencia

1) Indexación: Una de las diferencias conceptuales más relevantes con los sistemas relacionales vistos en la materia es que Spark **no mantiene estructuras de índices B-Tree o índices Hash secundarios persistentes** sobre las tablas. En volúmenes de cientos de terabytes, mantener índices B-Tree para cada registro introduce un sobrecosto de almacenamiento y actualización inmanejable para un motor analítico. En su lugar, Spark optimiza el acceso masivo mediante:

- *Poda de particiones* (*Partition Pruning*): Al estructurar el almacenamiento en carpetas (ej. `anio=2026/mes=11/`), Catalyst omite por completo escanear los directorios que no coinciden con las cláusulas `WHERE`.

- *Salto de datos* (*Data Skipping*): En archivos Parquet/ORC, cada bloque de registros (*Row Group*) almacena estadísticas de metadatos con los valores mínimo y máximo de cada columna. Si el valor buscado no pertenece al intervalo $[\min, \max]$, el bloque entero se descarta sin leerlo de disco.
- *Filtros de Bloom en tiempo de ejecución*: En operaciones de unión (*join*), Spark puede generar un filtro de Bloom compacto sobre la tabla menor para descartar tempranamente filas de la tabla mayor en los ejecutores antes de enviarlas al *shuffle*.

2) Modelo de consistencia: Spark es un motor computacional desacoplado del almacenamiento, no una base de datos transaccional con bloqueos a nivel de fila:

- *Consistencia de lectura*: Provee un aislamiento equivalente a *Snapshot Isolation* de facto: una consulta Spark lee una foto estática y consistente de los archivos existentes en el almacenamiento en el instante de iniciar la etapa de lectura.
- *Consistencia de escritura*: Para evitar que tareas fallidas dejen datos corruptos o duplicados, Spark emplea protocolos de confirmación atómica (`FileCommitProtocol`). Cada tarea escribe sus particiones en directorios de trabajo temporales (`_temporary`); únicamente cuando la etapa completa finaliza exitosamente, el *driver* renombra y confirma atómicamente los archivos en la ruta definitiva.
- *Garantías ACID sobre lagos de datos*: En arquitecturas modernas, Spark se integra con formatos transaccionales abiertos como **Delta Lake** [12] o Apache Iceberg. Estas capas incorporan registros de transacciones basados en bitácoras (*transaction logs*) con control de concurrencia optimista (OCC), dotando a Spark de operaciones ACID completas (`UPDATE`, `DELETE`, `MERGE` y viajes en el tiempo */time-travel*).

III. Casos de uso y comparación con el modelo relacional

A. Comparación fundamentada con el modelo relacional

El modelo relacional tradicional, formalizado por E. F. Codd en 1970, se diseñó bajo el paradigma de operaciones transaccionales de grano fino (OLTP, *Online Transaction Processing*). En la materia se estudian sus pilares: tablas bidimensionales, tuplas, dominios, claves primarias y foráneas con integridad referencial estricta, y el proceso de **normalización** (1FN, 2FN, 3FN, BCNF) orientado a erradicar redundancias y anomalías de inserción, modificación y borrado.

En contraste, Apache Spark responde al paradigma analítico masivo (OLAP, *Online Analytical Processing*):

- *Normalización frente a Desnormalización*: En un motor relacional, los datos se normalizan para ahorrar almacenamiento y garantizar consistencia puntual; las consultas recomponen la información mediante `JOIN` locales acelerados por índices B-Tree. En un sistema distribuido como Spark, las uniones entre tablas grandes distribuidas exigen redistribuir físicamente los datos por red (*shuffle*), lo que constituye la operación más costosa del clúster. Por ello, en Big Data se favorece deliberadamente la **desnormalización** y estructuras anidadas (registros complejos, arreglos, esquemas en estrella o copo de nieve) para procesar consultas masivas con escaneos columnares directos.
- *Transacciones ACID frente a Cómputo por Lotes*: Los sistemas relacionales emplean mecanismos complejos de concurrencia (bloqueos de dos fases 2PL, registros de escritura anticipada WAL) para procesar miles de transacciones concurrentes por segundo que modifican una fila a la vez. Spark no compite en este terreno: procesa lotes de millones o miles de millones de registros, operando sobre colecciones inmutables donde las escrituras se realizan por reemplazo o anexión (*append*) de particiones completas.
- *Escalabilidad*: Los RDBMS tradicionales escalan predominantemente de forma vertical; particionarlos horizontalmente (*sharding*) requiere lógica compleja a nivel de aplicación y degrada la integridad referencial. Spark escala horizontalmente de manera nativa y transparente.

La Tabla I resume la confrontación paradigmática.

TABLA I
Comparación entre bases de datos relacionales y Apache Spark

Dimensión	**Base de datos relacional**	**Apache Spark**
Propósito central	Transacciones OLTP y consultas operacionales	Procesamiento analítico OLAP y ML distribuido
Estructuración	Tablas normalizadas (1FN–3FN), integridad referencial	DataFrames desnormalizados, esquemas columnares
Indexación	Índices B-Tree / Hash secundarios en disco	Poda de particiones, data skipping y Bloom filters
Patrón de acceso	Lecturas/escrituras atómicas puntuales (< 10 ms)	Escaneo masivo de lotes y transformaciones en RAM
Escalabilidad	Vertical predominante (o sharding complejo)	Horizontal elástica sobre clústeres de cómputo
Garantías ACID	Integridad referencial y transacciones nativas	Dependientes de la capa de storage (ej. Delta Lake)
Uso de SQL	Consultas sobre esquemas fijos indexados	Optimización de planes lógicos/físicos con Catalyst

B. Casos de uso reales

Spark resulta la tecnología idónea en los siguientes escenarios industriales:

1) **Canalizaciones de Ingesta y ETL/ELT masivas**: Empresas del sector financiero y telecomunicaciones utilizan Spark para extraer cientos de gigabytes o terabytes diarios de transacciones, aplicar transformaciones complejas, desduplicación y validaciones de esquema, almacenándolos en lagos de datos para consumo corporativo.
2) **Entrenamiento distribuido de Machine Learning**: Modelos de segmentación de usuarios (K-Means), sistemas de recomendación masivos para comercio electrónico (factor-

ización matricial ALS) y clasificación de riesgo crediticio con Spark MLlib [6].

3) **Análisis interactivo de Big Data (OLAP)**: Consultas analíticas complejas sobre miles de millones de registros mediante Spark SQL conectado a plataformas de Business Intelligence (Tableau, PowerBI).
4) **Procesamiento continuo de flujos (*Structured Streaming*)**: Detección de fraude en transacciones de tarjetas de crédito y procesamiento de eventos IoT con ventanas temporales deslizantes.

C. Situaciones en las que no conviene utilizar Spark

Identificar cuándo **no** debe utilizarse Spark es un criterio ingenieril esencial:

- **Cargas transaccionales operativas (OLTP)**: Si la aplicación requiere actualizar o consultar saldos bancarios, registrar reservas de pasajes o gestionar carritos de compra con miles de transacciones concurrentes por segundo y latencias inferiores a 10 milisegundos, una base de datos relacional como PostgreSQL o MySQL es infinitamente superior.
- **Consultas puntuales indexadas por clave**: Consultar el perfil de un cliente por su clave primaria (`WHERE id = 12345`) en Spark requiere planificar tareas y escanear metadatos, tardando cientos de milisegundos o segundos; un RDBMS o una base clave–valor (Redis, Cassandra) lo resuelve en 1 milisegundo mediante índices B-Tree o tablas hash.
- **Conjuntos de datos que caben en una sola máquina**: Cuando los datos ocupan menos de 10 a 20 gigabytes, bibliotecas mono-nodo como Pandas, DuckDB, Polars o PostgreSQL son órdenes de magnitud más rápidas y simples. Distribuir datos pequeños en Spark agrega costos de serialización, coordinación del *driver* y red que ralentizan la ejecución innecesariamente.

IV. Operaciones concretas: Consultas, escrituras y agrupamiento iterativo

A. Consultas analíticas y escrituras particionadas

Para ilustrar las capacidades operacionales de la tecnología, el Listado 1 presenta un programa completo en PySpark que demuestra tres operaciones fundamentales: carga estructurada desde almacenamiento columnar Parquet, consultas analíticas con la API de DataFrames y con sintaxis ANSI SQL, y escritura particionada en almacenamiento distribuido.

Listado 1. Consultas analíticas y escrituras particionadas con PySpark

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, count, year
spark = SparkSession.builder.appName("OperacionesAnaliticas
    ").getOrCreate()

# 1. Lectura estructurada desde formato columnar
df = spark.read.parquet("hdfs:///datos/ventas.parquet")

# 2. Consulta y agregacion con DataFrame API
resumen_df = df.filter(col("monto") > 50.0) \
    .withColumn("anio", year(col("fecha"))) \
    .groupBy("anio", "categoria") \
    .agg(avg("monto").alias("monto_prom"), count("id").
     alias("total"))

# 3. Consulta equivalente con Spark SQL ANSI
df.createOrReplaceTempView("ventas")
resumen_sql = spark.sql("""
    SELECT year(fecha) AS anio, categoria,
           AVG(monto) AS monto_prom, COUNT(id) AS total
    FROM ventas WHERE monto > 50.0 GROUP BY 1, 2
""")

# 4. Escritura distribuida particionada en disco
resumen_df.write.mode("overwrite").partitionBy("anio") \
    .parquet("hdfs:///datos/salida/resumen_ventas/")
```

B. K-Means distribuido: Formulación algorítmica

Dado un conjunto de n puntos $x_1, \ldots, x_n \in \mathbb{R}^d$ y un número k de grupos, K-Means busca hallar k centroides $\mu_1, \ldots, \mu_k \in \mathbb{R}^d$ que minimicen la función de costo WSSSE (*Within-Set Sum of Squared Errors*):

$$J = \sum_{i=1}^{n} \min_{j \in \{1,\ldots,k\}} \|x_i - \mu_j\|^2. \tag{1}$$

El algoritmo de Lloyd [8], [9] aproxima numéricamente la solución alternando iterativamente dos fases hasta la convergencia:

1) **Asignación**: Cada punto x_i se clasifica al centroide más cercano: $c(i) = \arg\min_j \|x_i - \mu_j\|^2$.
2) **Actualización**: Se recalculan los centroides como el promedio vectorial de los puntos asignados: $\mu_j = \frac{1}{|C_j|} \sum_{i \in C_j} x_i$.

C. Paralelización mediante estadísticos suficientes

La fase de asignación es puramente local en cada ejecutor. La fase de actualización, en cambio, requiere consolidar la información de todas las particiones del clúster. Enviar todos los puntos al *driver* anularía la computación distribuida. Para evitarlo, cada ejecutor p calcula para cada grupo j un vector de suma acumulada $S_{p,j} \in \mathbb{R}^d$ y un contador $N_{p,j} \in \mathbb{N}$:

$$S_{p,j} = \sum_{i \in p,\, c(i)=j} x_i, \qquad N_{p,j} = \big|\{i \in p : c(i) = j\}\big|. \tag{2}$$

El par (S, N) constituye un **estadístico suficiente**: permite calcular la media exacta sin retener los puntos individuales. Como la suma vectorial y escalar son asociativas y conmutativas, los ejecutores reducen sus estadísticos y el *driver* divide una sola vez:

$$\mu'_j = \frac{\sum_p S_{p,j}}{\sum_p N_{p,j}}. \tag{3}$$

El volumen transferido por partición es solo de $k(d+1)$ números en punto flotante, completamente independiente del volumen n de registros.

D. El sesgo del promedio de promedios

Calcular en cada partición el vector promedio local $\bar{x}_{p,j} = S_{p,j}/N_{p,j}$ y promediar esos vectores en el *driver* genera un error matemático grave cuando las particiones poseen cardinalidades desiguales. Considérese un punto en $x = 10$ en la partición 1 y tres puntos en $x = 0$ en la partición 2:

$$\underbrace{\frac{10+0}{2} = 5}_{\text{Promedio de promedios (erróneo)}} \quad \neq \quad \underbrace{\frac{10+0+0+0}{4} = 2{,}5}_{\text{Promedio real ponderado}}. \tag{4}$$

TABLA II
ESTRATEGIAS DE COMUNICACIÓN ENTRE EXECUTORS Y DRIVER

Estrategia	Contenido	Volumen	Exacto
A (Centralizada)	Puntos completos de partición	$\propto n_p$ (19 MiB/10^6)	Sí
B (Promedio local)	Promedio local $\bar{x}_{p,j}$	48 B	No
C (Estadístico suf.)	Suma $S_{p,j}$ y conteo $N_{p,j}$	72 B	Sí

La Tabla II compara las estrategias de comunicación para $k = 3$, $d = 2$ y valores de 8 bytes (doble precisión).

E. Implementación de K-Means en PySpark MLlib

Spark implementa la inicialización paralelizable *k-means‖* [10]. El Listado 2 exhibe el código para entrenar K-Means con MLlib.

Listado 2. K-Means con PySpark y MLlib

```
from pyspark.ml.clustering import KMeans
from pyspark.ml.feature import VectorAssembler
df = spark.read.csv("clientes.csv", header=True,
     inferSchema=True)
vec = VectorAssembler(inputCols=["x", "y"], outputCol="
     features")
datos = vec.transform(df).cache()
modelo = KMeans(k=3, maxIter=12, seed=1).fit(datos)
print(modelo.clusterCenters())
print(modelo.summary.trainingCost)  # WSSSE
```

V. TOLERANCIA A FALLOS Y RENDIMIENTO

A. Recuperación por linaje

En clústeres a gran escala compuestos por decenas o cientos de nodos, las fallas transitorias o definitivas de hardware son eventos esperados. Hadoop mitiga este fenómeno replicando bloques de datos tres veces en disco local [2]. Spark, en contrapartida, evita la costosa replicación en memoria de estructuras intermedias apoyándose en el linaje del RDD [4]. Cada RDD mantiene información de solo lectura sobre sus transformaciones ascendentes. Si un nodo ejecutor cae durante la iteración 10 de K-Means, el *driver* detecta la pérdida de la tarea y la reasigna a otro ejecutor disponible, quien reconstruye exclusivamente la partición afectada releyendo su porción de la fuente base persistente.

B. Tareas rezagadas y ejecución especulativa

La velocidad de una etapa paralela está gobernada por su tarea más lenta (*straggler*). Los rezagados surgen por degradación de discos locales, contención de red o competencia de hilos. Dean y Ghemawat propusieron neutralizar este cuello de botella despachando tareas duplicadas de respaldo (*backup tasks*) [1]. Spark implementa esta estrategia mediante el parámetro `spark.speculation`. Cuando se habilita, el sistema monitorea la duración de las tareas y relanza una copia especulativa en otro nodo si una tarea en ejecución supera un múltiplo configurado (`spark.speculation.multiplier`) de la mediana de su etapa, adoptando los resultados de la primera instancia que finalice con éxito [16].

TABLA III
COMPARACIÓN ENTRE HADOOP MAPREDUCE Y APACHE SPARK

Característica	Hadoop MapReduce	Apache Spark
Datos entre pasos	Disco distribuido (HDFS)	Memoria RAM (`cache`)
Modelo de cómputo	Map y Reduce acoplados	Grafo acíclico (DAG) flexible
Tolerancia a fallos	Replicación física de bloques	Linaje determinista y recálculo
Soporte iterativo	Un trabajo aislado por paso	Un único contexto de ejecución
APIs soportadas	Java	Python, Scala, SQL, R, Java

C. Comparación analítica entre Spark y Hadoop

Sea r el costo temporal de leer una partición desde disco, c el tiempo de cálculo local por iteración y w el tiempo de volcar resultados intermedios a almacenamiento persistente. Para un algoritmo que requiere m iteraciones sucesivas, el costo acumulado de Hadoop MapReduce se modela como:

$$T_{\text{Hadoop}} \approx m\,(r + c + w), \tag{5}$$

mientras que Spark, preservando los datos en memoria tras la primera lectura, requiere:

$$T_{\text{Spark}} \approx r + m\,c. \tag{6}$$

En aplicaciones iterativas donde la I/O domina sobre el cómputo ($r + w \gg c$), la aceleración teórica crece casi proporcionalmente a m. Investigaciones empíricas reportan reducciones de tiempo de hasta un orden de magnitud a favor de Spark en algoritmos de aprendizaje automático [4]. La Tabla III sintetiza las diferencias paradigmáticas.

VI. MACHINE LEARNING A ESCALA: ARQUITECTURA, PIPELINES Y OPTIMIZACIÓN

La paralelización básica descrita en la Sección IV asume una comunicación plana hacia el maestro. Sin embargo, en despliegues con petabytes de datos y miles de particiones, la arquitectura debe resolver problemas de saturación de red, orquestación de características, optimización de hardware y asimetría de particiones.

A. Comunicación distribuida y agregación en árbol

Cuando un clúster opera con miles de particiones P, enviar los vectores de estadísticos $(S_{p,j}, N_{p,j})$ directamente al *driver* origina un cuello de botella $O(P)$ de conexiones y tráfico de red entrante sobre un único puerto, desencadenando agotamiento de memoria (*Out Of Memory*, OOM) y serialización forzada de las adiciones.

Para resolver esta vulnerabilidad, Spark MLlib implementa la **agregación jerárquica en árbol** (`treeAggregate` y `treeReduce`) [6]. En lugar de transferir todos los mensajes a un punto central, la reducción se estructura en niveles sucesivos con una profundidad configurable $h \geq 2$ (`depth`), como se ilustra en la Fig. 2.

En el nivel 0, cada ejecutor consolida las estadísticas de sus particiones locales. En los niveles intermedios, un grupo de

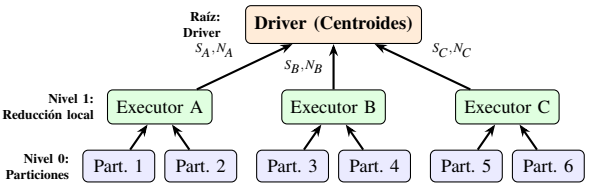


Fig. 2. Agregación jerárquica con `treeAggregate` en Spark MLlib. Los ejecutores intermedios combinan estadísticos suficientes en paralelo antes de transmitirlos al nodo maestro, evitando saturar la interfaz de red del *driver*.

ejecutores actúa como agregador de un factor de ramificación $b = \lceil P^{1/h} \rceil$ de particiones hijas. Este esquema garantiza que:

- La cantidad de mensajes que ingresan al *driver* disminuye desde $O(P)$ hasta $O(P^{1/h})$.
- El cómputo aritmético de suma vectorial se ejecuta en paralelo entre múltiples núcleos distribuidos.
- La latencia total de agregación escala de manera logarítmica respecto al volumen de particiones.

B. Canalizaciones de Machine Learning (ML Pipelines)

En la práctica industrial de macrodatos, la fase de ajuste del modelo suele representar menos del 20 % del cómputo; el preprocesamiento, escalamiento, imputación y codificación de atributos acaparan el resto de la carga. La API basada en DataFrames (`pyspark.ml`) estandariza estos flujos de trabajo mediante tres abstracciones fundamentales:

- **Transformer**: Algoritmo que recibe un DataFrame y produce un nuevo DataFrame derivado (añadiendo columnas de características, vectores densos o predicciones). Operan aplicando transformaciones fila por fila con dependencias estrechas, procesándose en paralelo local sin originar transferencias por *shuffle*.
- **Estimator**: Algoritmo parametrizado que implementa el método `fit()` sobre un DataFrame para aprender patrones de los datos, produciendo como resultado un *Transformer* entrenado.
- **Pipeline**: Grafo lineal ordenado que encadena múltiples transformadores y estimadores. Garantiza la consistencia del flujo, erradica la fuga de datos (*data leakage*) entre conjuntos de entrenamiento y evaluación y facilita la exportación atómica del modelo a producción.

El Listado 3 muestra una canalización distribuida completa que realiza ensamblado vectorial, escalamiento y entrenamiento de K-Means, junto con su posterior evaluación de calidad mediante el coeficiente Silhouette.

Listado 3. Canalización completa de Machine Learning con PySpark

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler,
    StandardScaler
from pyspark.ml.clustering import KMeans
from pyspark.ml.evaluation import ClusteringEvaluator
# Ingestion columnar Parquet y preparacion distribuida
df = spark.read.parquet("hdfs:///datos/clientes.parquet")
vec = VectorAssembler(inputCols=["ingreso", "antiguedad", "
    gastos"], outputCol="vec")
std = StandardScaler(inputCol="vec", outputCol="features",
    withMean=True, withStd=True)
# Definicion y ajuste del Pipeline con estimador KMeans
kmeans = KMeans(k=4, initMode="k-means||", maxIter=25, seed
    =42)
pipeline = Pipeline(stages=[vec, std, kmeans])
modelo = pipeline.fit(df)
# Evaluacion distribuida con metrica Silhouette
pred = modelo.transform(df)
score = ClusteringEvaluator(metricName="silhouette").
    evaluate(pred)
print(f"Silhouette = {score:.4f}")
```

C. El motor de optimización: Catalyst y Project Tungsten

El elevado rendimiento de Spark sobre grandes volúmenes de datos descansa sobre dos componentes internos de su motor relacional y analítico [11]:

Catalyst Optimizer: Motor de optimización extensible que aplica álgebra relacional sobre árboles sintácticos de operaciones en cuatro etapas consecutivas:

1) *Análisis*: Resuelve identificadores de columnas, tipos y esquemas contra el catálogo.
2) *Optimización lógica*: Aplica reglas canónicas deterministas, destacando el empuje de predicados (*predicate pushdown*) que filtra registros directamente en el formato de almacenamiento (ej. omitiendo bloques en archivos Parquet sin cargarlos a memoria) y la poda de proyecciones (*column pruning*) para ignorar columnas redundantes.
3) *Optimización física*: Evalúa planes de ejecución mediante modelos de costo (CBO), seleccionando automáticamente estrategias eficientes como transformar uniones costosas en *Broadcast Hash Joins* cuando una tabla es pequeña, eliminando la etapa de *shuffle*.
4) *Generación de código*: Traduce partes del plan optimizado a *bytecode* nativo de Java para su ejecución inmediata.

Project Tungsten: Rediseño de bajo nivel enfocado en optimizar el cómputo a nivel de CPU y memoria física, eliminando la sobrecarga intrínseca de la máquina virtual Java (JVM):

- *Memoria Off-Heap y formato binario*: Spark gestiona memoria en formato compacto (`UnsafeRow`), eliminando encabezados de objetos Java (16–24 bytes por tupla) y pausas de recolección de basura (*GC pauses*).
- *Whole-Stage Code Generation*: Sintetiza en tiempo de ejecución bucles cerrados en Java plano que compilan cadenas completas de operadores, manteniendo datos en los registros y cachés L1/L2/L3 del procesador.

D. Desafíos operativos: Sesgo de datos y gestión de Shuffle

En entornos productivos, el rendimiento del clúster puede degradarse drásticamente si no se gestionan adecuadamente los cuellos de botella de particionamiento [17]:

Sesgo de datos (*Data Skew*): Se manifiesta cuando una distribución no uniforme en las claves de particionamiento concentra una proporción masiva de registros en un único nodo. El ejecutor afectado se convierte en un cuello de botella que retarda la culminación de toda la etapa paralela. Para contrarrestarlo se aplican dos técnicas:

1) *Salting*: Adición de un prefijo o sufijo aleatorio uniforme $s \in [0, S-1]$ a las claves problemáticas, permitiendo repartir

los registros en S subparticiones durante la primera agregación y unificar los resultados mediante una reducción posterior.

2) *Adaptive Query Execution (AQE)*: Característica introducida en versiones recientes que analiza estadísticas en tiempo de ejecución al concluir cada etapa de *shuffle*, subdividiendo automáticamente particiones anómalas sin requerir reescritura manual del código.

Desbordamiento a disco (*Shuffle Spill*): Durante las etapas de *shuffle*, los ejecutores ordenan y reagrupan grandes volúmenes de datos. Si la memoria asignada a la ejecución (`spark.memory.fraction`) es desbordada, Spark escribe temporalmente los fragmentos excedentes a disco local sin compresión (*spill to disk*), penalizando la velocidad entre un factor de 10x y 50x. Se previene calibrando el número de particiones (`spark.sql.shuffle.partitions`, buscando bloques de 100 a 200 MiB) y configurando el serializador binario Kryo (`KryoSerializer`) en sustitución de la serialización nativa de Java.

VII. Conclusiones

El procesamiento distribuido sobre macrodatos constituye una necesidad técnica cuando el volumen, la velocidad y la dimensionalidad de la información desbordan la capacidad de un servidor individual. Apache Spark superó las limitaciones de I/O en disco del modelo MapReduce al introducir colecciones inmutables en memoria con tolerancia a fallos basada en linaje determinista, posibilitando la ejecución eficiente de algoritmos iterativos.

El análisis de su arquitectura y modelo de datos reveló que el éxito de Spark radica en el desacoplamiento del cómputo respecto al almacenamiento: prescinde de índices B-Tree tradicionales y replicación en memoria para maximizar el rendimiento analítico mediante poda de particiones, escaneo columnar (*data skipping*) y optimizaciones de bajo nivel con Catalyst y Project Tungsten. El caso de estudio de K-Means demostró que el escalamiento horizontal efectivo depende del empleo de estadísticos suficientes y de esquemas de agregación jerárquica (`treeAggregate`) que neutralizan los cuellos de botella en el nodo maestro.

Asimismo, se evidenció que los sistemas de gestión de bases de datos relacionales y los motores analíticos como Spark no son tecnologías excluyentes sino complementarias. Mientras que los primeros garantizan consistencia transaccional ACID en operaciones de negocio operacionales (OLTP) de grano fino, Spark provee la infraestructura analítica masiva y elástica (OLAP) requerida para la ingeniería de datos y el entrenamiento de modelos de aprendizaje automático a gran escala.

Finalmente, la consecución de una alta eficiencia en producción exige resolver desafíos de infraestructura tales como el diseño de esquemas de particionamiento balanceados, la mitigación del sesgo de datos (*data skew*) y el control del desbordamiento en el *shuffle*, confirmando la necesidad de un enfoque integral de ingeniería de datos para alcanzar escalabilidad lineal en el procesamiento masivo.

Referencias

[1] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, San Francisco, CA, USA, 2004, pp. 137–150.

[2] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST)*, 2010, pp. 1–10.

[3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster computing with working sets," in *Proc. 2nd USENIX Workshop Hot Topics in Cloud Computing (HotCloud)*, 2010.

[4] M. Zaharia *et al.*, "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in *Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, San Jose, CA, USA, 2012, pp. 15–28.

[5] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Commun. ACM*, vol. 59, no. 11, pp. 56–65, 2016.

[6] X. Meng *et al.*, "MLlib: Machine learning in Apache Spark," *J. Mach. Learn. Res.*, vol. 17, no. 34, pp. 1–7, 2016.

[7] V. K. Vavilapalli *et al.*, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. 4th ACM Symp. Cloud Computing (SoCC)*, 2013, pp. 5:1–5:16.

[8] S. Lloyd, "Least squares quantization in PCM," *IEEE Trans. Inf. Theory*, vol. 28, no. 2, pp. 129–137, 1982.

[9] J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. 5th Berkeley Symp. Mathematical Statistics and Probability*, vol. 1, 1967, pp. 281–297.

[10] B. Bahmani, B. Moseley, A. Vattani, R. Kumar, and S. Vassilvitskii, "Scalable k-means++," *Proc. VLDB Endowment*, vol. 5, no. 7, pp. 622–633, 2012.

[11] M. Armbrust *et al.*, "Spark SQL: Relational data processing in Spark," in *Proc. 2015 ACM SIGMOD Int. Conf. Management of Data*, Melbourne, Australia, 2015, pp. 1383–1394.

[12] M. Armbrust *et al.*, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411–3424, 2020.

[13] The Apache Software Foundation, "Cluster mode overview," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/cluster-overview.html (accedido: nov. 2026).

[14] The Apache Software Foundation, "Clustering — K-means," *Apache Spark MLlib Guide*. [Online]. Available: https://spark.apache.org/docs/latest/ml-clustering.html (accedido: nov. 2026).

[15] The Apache Software Foundation, "Spark SQL, DataFrames and Datasets Guide," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/sql-programming-guide.html (accedido: nov. 2026).

[16] The Apache Software Foundation, "Spark configuration," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/configuration.html (accedido: nov. 2026).

[17] The Apache Software Foundation, "Tuning Spark," *Apache Spark Documentation*. [Online]. Available: https://spark.apache.org/docs/latest/tuning.html (accedido: nov. 2026).

[18] The PostgreSQL Global Development Group, "Transactions," *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/current/tutorial-transactions.html (accedido: nov. 2026).